PHASE 6 — Recruitment (ATS) & Performance Management

PRSECURITY HRMS

IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.
After each file write "--- FILE COMPLETE ---" then start the next file immediately.
If about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"
This is Phase 6 of 8. Phases 1–5 done: Auth, Employees, Dashboards, Attendance, Leave, Payroll, Tasks, Projects, Clients.
DO NOT rebuild previous files. Only add NEW files or EXTEND where specified.

CONTEXT

Project: PRSECURITY HRMS | Stack: React+Vite+TS+Tailwind+shadcn/ui | Node+Express+Prisma+PostgreSQL Roles: ADMIN, HR, EMPLOYEE, INTERN | Currency: ₹

PART A — RECRUITMENT / ATS

STEP A1 — Update Prisma Schema

prisma

```
model JobPosting {
  id              String      @id @default(uuid())
  title           String
  description     String
  requirements    String
  responsibilities String?
  location        String?
  type            EmploymentType @default(FULL_TIME)
  status          JobStatus    @default(OPEN)
  salaryMin       Float?
  salaryMax       Float?
  openings        Int         @default(1)
  closingDate     DateTime?
  createdBy       String
  creator         User        @relation(fields: [createdBy], references: [id])
  applicants      Applicant[]
  createdAt       DateTime    @default(now())
  updatedAt       DateTime    @updatedAt
}

enum JobStatus { OPEN CLOSED ON_HOLD }

model Applicant {
  id              String      @id @default(uuid())
  jobPostingId    String
  jobPosting      JobPosting  @relation(fields: [jobPostingId], references: [id])
  firstName       String
  lastName        String
  email           String
  phone           String?
  resumeUrl       String?
  coverLetter     String?
  currentCompany  String?
  currentDesignation String?
  noticePeriod    String?
  expectedSalary  Float?
  status          ApplicantStatus @default(APPLIED)
  notes           String?
  interviews      Interview[]
  createdAt       DateTime    @default(now())
  updatedAt       DateTime    @updatedAt
}

enum ApplicantStatus {
```

```
  APPLIED
  SHORTLISTED
  INTERVIEW_SCHEDULED
  OFFERED
  HIRED
  REJECTED
}

model Interview {
  id          String   @id @default(uuid())
  applicantId String
  applicant   Applicant @relation(fields: [applicantId], references: [id])
  scheduledAt DateTime
  mode        String   @default("IN_PERSON")
  feedback    String?
  rating      Int?
  result      String?
  interviewers InterviewerOnInterview[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model InterviewerOnInterview {
  id          String   @id @default(uuid())
  interviewId String
  userId      String
  interview   Interview @relation(fields: [interviewId], references: [id])
  user        User     @relation(fields: [userId], references: [id])
  @@unique([interviewId, userId])
}
```

Run: `npx prisma migrate dev --name add_recruitment`

STEP A2 — Recruitment Backend

FILE: backend/src/services/recruitment.service.ts

`getAllJobPostings(query)`:
Paginated, filter by status
Include applicant count per posting

`createJobPosting(data, createdBy)`:
Create job posting

`getJobPosting(id)`:
Full detail with applicants grouped by status (pipeline stages)

`updateJobPosting(id, data)`:
Update fields

`updateJobStatus(id, status)`:

Open/Close/Hold a posting

`addApplicant(jobPostingId, data, resumeFile?)`:

Create applicant record

If resumeFile: save via multer, store path

Notify HR/Admin: "New applicant {name} for {job title}"

`getApplicant(id)`:

Full applicant detail with interview history

`updateApplicantStatus(id, status, note?)`:

Move pipeline stage

If status=HIRED: don't auto-create employee yet (separate action)

`scheduleInterview(applicantId, data)`:

Create Interview record

Update applicant status to INTERVIEW_SCHEDULED

Notify interviewers

`updateInterview(id, data)`:

Add feedback, rating, result

`hireApplicant(applicantId, hrUserId)`:

Get applicant data

Create new User account with role=EMPLOYEE

Auto-generate employeeId

Set temp password = Prs@{employeeId}

Create basic SalaryStructure (basicSalary=0, to be updated by HR)

Update applicant status=HIRED

Send notification to HR/Admin: "{name} hired and added as employee {employeeId}"

Return newly created user

`getRecruitmentStats()`:

Total open jobs, total applicants this month, total hired this month, pipeline counts by stage

FILE: backend/src/controllers/recruitment.controller.ts

FILE: backend/src/routes/recruitment.routes.ts

```
GET    /api/jobs                  → getAllJobPostings (HR/Admin)
POST   /api/jobs                  → createJobPosting (HR/Admin)
GET    /api/jobs/stats            → getRecruitmentStats (HR/Admin)
GET    /api/jobs/:id              → getJobPosting (HR/Admin)
PUT    /api/jobs/:id              → updateJobPosting (HR/Admin)
PUT    /api/jobs/:id/status       → updateJobStatus (HR/Admin)
POST   /api/applicants            → addApplicant (HR/Admin, multipart for resume)
GET    /api/applicants/:id        → getApplicant (HR/Admin)
PUT    /api/applicants/:id/status → updateApplicantStatus (HR/Admin)
POST   /api/applicants/:id/interview → scheduleInterview (HR/Admin)
PUT    /api/interviews/:id        → updateInterview (HR/Admin)
POST   /api/applicants/:id/hire   → hireApplicant (HR/Admin)
```

STEP A3 — Recruitment Frontend

FILE: frontend/src/api/recruitment.api.ts

FILE: frontend/src/pages/recruitment/RecruitmentPage.tsx

PageHeader: "Recruitment" + "Post New Job" button
StatCards: Open Positions, Applicants This Month, Hired This Month, Interviews This Week
Job Postings list:
Each row: Job Title, Type badge, Openings, Applicant Count, Status badge, Closing Date, Actions (View Pipeline, Edit, Close)
Filter by status (Open/Closed/On Hold)

FILE: frontend/src/pages/recruitment/JobDetailPage.tsx

Back button, Job title, type, status badge, salary range (₹), closing date
Job description, requirements, responsibilities (rendered as formatted text)
Edit Job button
"Add Applicant" button → opens AddApplicantModal
Applicant Pipeline Kanban:
6 columns: Applied | Shortlisted | Interview Scheduled | Offered | Hired | Rejected
Each applicant card: Name, current company, expected salary (₹), applied date
Drag card between columns to update status
Click card → opens ApplicantDetailDrawer

FILE: frontend/src/components/recruitment/ApplicantDetailDrawer.tsx

Right-side drawer:
Applicant info: name, email, phone, current company/designation, notice period, expected salary (₹)
Resume download button (if uploaded)
Cover letter section
Status dropdown (update pipeline stage)
Notes textarea (save notes)
Interview History:
List of past interviews: date, mode, interviewers, rating (stars), result badge
"Schedule Interview" button → ScheduleInterviewModal
"Hire" button (only if status=OFFERED) → confirm modal → calls hireApplicant → shows new employee ID + temp password

FILE: frontend/src/components/recruitment/AddApplicantModal.tsx

Form: First Name, Last Name, Email, Phone, Current Company, Current Designation, Notice Period, Expected Salary (₹), Cover Letter textarea, Resume upload (PDF only), Notes.

FILE: frontend/src/components/recruitment/ScheduleInterviewModal.tsx

Form: Date & Time picker, Mode (In Person/Video/Phone), Select Interviewers (multi-select from employee list).

FILE: frontend/src/components/recruitment/JobFormModal.tsx

Create/Edit job posting form:
Title, Description (textarea), Requirements (textarea), Responsibilities (textarea)
Location, Employment Type dropdown, Openings count, Salary Min (₹), Salary Max (₹), Closing Date

PART B — PERFORMANCE MANAGEMENT

STEP B1 — Update Prisma Schema

prisma

```
model PerformanceGoal {
  id          String     @id @default(uuid())
  userId      String
  title       String
  description String?
  targetDate  DateTime?
  weight      Float      @default(0)
  status      GoalStatus @default(ACTIVE)
  createdBy   String
  user        User       @relation(fields: [userId], references: [id])
  creator     User       @relation("GoalCreator", fields: [createdBy], references: [id])
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
}

enum GoalStatus { ACTIVE COMPLETED MISSED }

model PerformanceReview {
  id             String       @id @default(uuid())
  userId         String
  reviewerId     String
  period         String
  year           Int
  selfRating     Float?
  managerRating  Float?
  selfComments   String?
  managerComments String?
  overallRating  Float?
  status         ReviewStatus @default(DRAFT)
  user           User         @relation(fields: [userId], references: [id])
  reviewer       User         @relation("ReviewerUser", fields: [reviewerId], references: [id])
  createdAt      DateTime     @default(now())
  updatedAt      DateTime     @updatedAt
}

enum ReviewStatus { DRAFT SUBMITTED COMPLETED }
```

Run: `npx prisma migrate dev --name add_performance`

STEP B2 — Performance Backend

FILE: backend/src/services/performance.service.ts

`getMyGoals(userId)`:
All goals for user with status filter

`createGoal(data, createdBy)`:
Create goal for userId
Notify user if created by manager

`updateGoal(id, data, userId, role)`:
User can update own goals; HR/Admin can update any

`deleteGoal(id, userId, role)`:
Only creator or Admin

`getMyReviews(userId)`:
All review cycles for user

`getAllReviews(query)`:
HR/Admin: all reviews, filter by year/period/status

`initiateReviewCycle(year, period, hrUserId)`:
period: Q1 | Q2 | Q3 | Q4 | ANNUAL
Create PerformanceReview records for ALL active employees
Set reviewer = their HR (for now, set reviewer = the HR initiating)
Status = DRAFT
Notify all employees: "Performance review for {period} {year} has been initiated"
Return count created

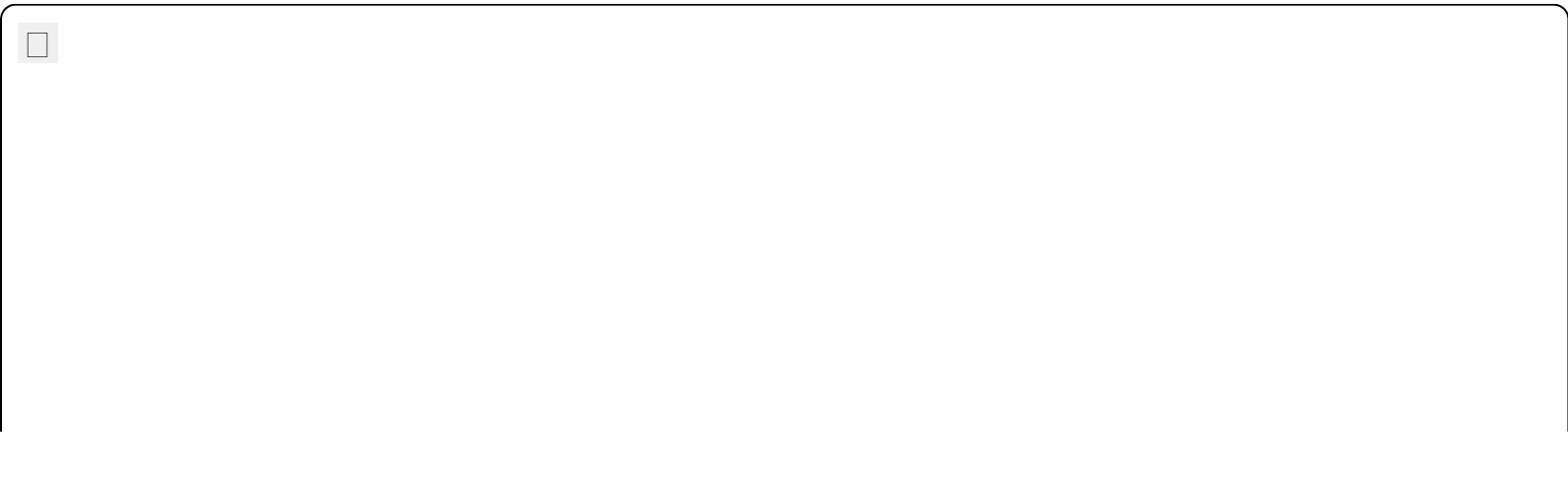
`submitSelfReview(reviewId, userId, data)`:
data: selfRating (1-5), selfComments
Validate review belongs to userId
Update + set status=SUBMITTED
Notify reviewer

`submitManagerReview(reviewId, reviewerId, data)`:
data: managerRating, managerComments, overallRating
Validate reviewerId matches
Update + set status=COMPLETED

`getReviewStats()`:
Count by status for current cycle

FILE: backend/src/controllers/performance.controller.ts

FILE: backend/src/routes/performance.routes.ts



```
GET    /api/performance/goals/my       → getMyGoals (authenticated)
POST   /api/performance/goals          → createGoal (authenticated)
PUT    /api/performance/goals/:id      → updateGoal (authenticated)
DELETE /api/performance/goals/:id      → deleteGoal (authenticated)
GET    /api/performance/reviews/my     → getMyReviews (authenticated)
GET    /api/performance/reviews        → getAllReviews (HR/Admin)
POST   /api/performance/reviews/initiate → initiateReviewCycle (HR/Admin)
PUT    /api/performance/reviews/:id/self → submitSelfReview (authenticated)
PUT    /api/performance/reviews/:id/manager → submitManagerReview (HR/Admin)
```

STEP B3 — Performance Frontend

FILE: frontend/src/api/performance.api.ts

FILE: frontend/src/components/performance/GoalCard.tsx

Card showing one goal:
Title, description, target date, weight badge, status badge
Progress bar (visual indicator)
Edit + Delete buttons

FILE: frontend/src/components/performance/ReviewCard.tsx

Card showing one review cycle:
Period + Year heading
Self Rating stars (if submitted)
Manager Rating stars (if completed)
Status badge
"Submit Self Review" button if status=DRAFT
"View Details" button

FILE: frontend/src/components/performance/SelfReviewForm.tsx

Form: Rating slider 1–5 (with star display), Comments textarea, Submit button.

FILE: frontend/src/components/performance/ManagerReviewForm.tsx

Form showing self review on left, manager inputs on right:
Left: employee self rating + comments (read-only)
Right: Manager Rating slider 1–5, Manager Comments textarea, Overall Rating slider, Submit button

FILE: frontend/src/pages/performance/PerformancePage.tsx

Employee/Intern view:
PageHeader: "My Performance"
Tab 1 — Goals:
"Add Goal" button → modal form: title, description, target date, weight (%)
Grid of GoalCards
Total weight indicator (should add to 100%)
Tab 2 — Reviews:
Grid of ReviewCards
Click card with status=DRAFT → opens SelfReviewForm in modal
Click completed card → shows both ratings side by side

FILE: frontend/src/pages/performance/PerformanceManagePage.tsx

HR/Admin view:

PageHeader: "Performance Management" + "Initiate Review Cycle" button
"Initiate Review Cycle" → modal: select Period (Q1/Q2/Q3/Q4/ANNUAL) + Year → confirm
Tab 1 — Review Cycles:
Table: Employee, Period, Year, Self Rating, Manager Rating, Overall Rating, Status
Filter by period/year/status
Click row → opens ManagerReviewForm in drawer (if submitted)
Tab 2 — Goals Overview:
Table: Employee, Goal Title, Weight, Target Date, Status
Filter by employee, status

AFTER ALL FILES ARE COMPLETE

Write this exactly: "✅ PHASE 6 COMPLETE — Recruitment ATS (job postings, applicant pipeline, interviews, hire-to-employee) and Performance Management (goals, review cycles, self + manager ratings) are ready. Proceed to Phase 7 for Assets, Expenses & Helpdesk."